

LAPORAN PRAKTIKUM STRUKTUR DATA

PEKAN 5



Oleh :

M.YAZEM AGVA ROIZ

NIM 2511533003

MATA KULIAH STRUKTUR DATA

DOSEN PENGAMPU : DR. WAHYUDI, S.T, M.T

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

KATA PENGANTAR

Pedoman ini disusun sebagai rujukan resmi bagi mahasiswa Departemen Informatika dalam penyusunan laporan praktikum pada mata kuliah Pemrograman Dasar dengan Java. Dokumen ini tidak hanya memberikan gambaran umum mengenai format penulisan, tetapi juga menguraikan secara rinci sistematika laporan, tata cara penyajian isi, serta contoh penulisan kode program yang dilengkapi dengan referensi ilmiah. Melalui panduan ini, mahasiswa diharapkan mampu menyusun laporan yang tidak sekadar memenuhi aspek administratif, tetapi juga mencerminkan ketelitian, keteraturan, dan penerapan kaidah penulisan akademik pada tingkat dasar. Dengan demikian, laporan praktikum yang dihasilkan dapat berfungsi sebagai media pembelajaran, dokumentasi kegiatan, sekaligus sarana untuk melatih keterampilan menulis ilmiah yang akan bermanfaat dalam jenjang studi selanjutnya.

Padang, 2026

Penulis

DAFTAR ISI

BAB 1.....	4
1. Latar Belakang.....	4
2. Tujuan.....	4
3. Manfaat.....	4
BAB 2.....	5
A. Teori.....	5
B. Code.....	5
C. TUGAS.....	17
BAB 3.....	33
a. Kesimpulan.....	33
b. Saran.....	33

BAB 1

PENDAHULUAN

1. Latar Belakang

Dalam pembuatan program diperlukannya sebuah struktur yang tepat untuk kasus tertentu, linked list merupakan salah satu struktur yang dapat diandalkan serta menjadi pondasi utama dalam banyak kasus.

2. Tujuan

Tujuan dari praktikum ini adalah mempelajari linked list dengan cara membuat codenya untuk mendapatkan pemahaman yang dalam serta melakukan validasi atas code yang dibuat untuk mendapatkan pemahaman yang benar.

3. Manfaat

Manfaat melakukan praktikum ini adalah mendapatkan pemahaman yang dalam dan benar serta melatih dalam menulis kode berbasis code linked list dengan benar dan tepat.

BAB 2

PEMBAHASAN

A. Teori

Linked list merupakan struktur data yang berbasis *Node* yang saling terhubung dengan *Node* lain secara dinamis dan berurutan. Linked list memiliki 2 struktur utama yaitu:

1. Data

Merupakan nilai yang disimpan pada setiap nodenya, nilai dimasukkan dengan cara membuat node baru lalu dihubungkan dengan node yang sudah ada yang akan membuat rantai.

2. Pointer

Pointer adalah sebuah link ke node yang ada didepan atau di belakangnya, berfungsi untuk mengakses node lainnya yang berada di rantai antar node.

Linked list akan membuat rantai pada setiap operasinya yang membuat linked-list sangatlah dinamis dan dapat digunakan pada kasus dengan ukuran yang tidak pasti.

B. Code

a. NodeSLL_2511533003

```
package pekan5_2511533003;

public class NodeSLL_2511533003 {

    int data_3003;

    NodeSLL_2511533003 next_3003;

    public NodeSLL_2511533003(int data_3003) {

        this.data_3003 = data_3003;

        this.next_3003 = null;
    }
}
```

```
}  
}
```

Code diatas merupakan ADT yang berfungsi sebagai node pada linked-list. Terdapat property data_3003 yang akan menyimpan value dan next_3003 berfungsi sebagai pointer yang akan mengarah ke node lainnya.

b. TambahSLL_2511533003

```
package pekan5_2511533003;  
  
public class TambahSLL_2511533003 {  
  
    public static NodeSLL_2511533003 insertAtFront_3003(  
        NodeSLL_2511533003 head_3003,  
        int value_3003  
    ) {  
        NodeSLL_2511533003 new_node_3003 = new  
NodeSLL_2511533003(value_3003);  
        new_node_3003.next_3003 = head_3003;  
        return new_node_3003;  
    }  
  
    public static NodeSLL_2511533003 insertAtEnd_3003(  
        NodeSLL_2511533003 head_3003,  
        int value_3003  
    ) {  
        NodeSLL_2511533003 newNode3003 = new NodeSLL_2511533003(value_3003);
```

```

    if (head_3003 == null) {
        return newNode3003;
    }

    NodeSLL_2511533003 last_3003 = head_3003;

    while (last_3003.next_3003 != null) {
        last_3003 = last_3003.next_3003;
    }

    last_3003.next_3003 = newNode3003;

    return head_3003;
}

static NodeSLL_2511533003 GetNode_3003(int data_3003) {
    return new NodeSLL_2511533003(data_3003);
}

static NodeSLL_2511533003 insertPos_3003(
    NodeSLL_2511533003 headNode_3003,
    int position_3003,
    int value_3003
) {
    NodeSLL_2511533003 head_3003 = headNode_3003;
    if (position_3003 < 1) System.out.println("Invalid Position");

```

```

if (position_3003 == 1) {
    NodeSLL_2511533003 new_node_3003 = new NodeSLL_2511533003(
        value_3003
    );
    new_node_3003.next_3003 = head_3003;
    return new_node_3003;
} else {
    while (position_3003-- != 0) {
        if (position_3003 == 1) {
            NodeSLL_2511533003 newNode_3003 = GetNode_3003(value_3003);
            newNode_3003.next_3003 = headNode_3003.next_3003;
            headNode_3003.next_3003 = newNode_3003;
            break;
        }
        headNode_3003 = headNode_3003.next_3003;
    }

    if (position_3003 != 1) System.out.println(
        "Posisi di luar jangkauan"
    );

    return head_3003;
}
}

public static void printList_3003(NodeSLL_2511533003 head_3003) {
    NodeSLL_2511533003 curr_3003 = head_3003;

```



```

while (curr_3003.next_3003 != null) {

    System.out.print(curr_3003.data_3003 + "-->");

    curr_3003 = curr_3003.next_3003;

}

if (curr_3003.next_3003 == null) {

    System.out.print(curr_3003.data_3003);

    System.out.println();

}

}

public static void main(String[] args) {

    NodeSLL_2511533003 head_3003 = new NodeSLL_2511533003(2);

    head_3003.next_3003 = new NodeSLL_2511533003(3);

    head_3003.next_3003.next_3003 = new NodeSLL_2511533003(5);

    head_3003.next_3003.next_3003.next_3003 = new NodeSLL_2511533003(6);


    System.out.print("Senarai berantai awal:");

    printList_3003(head_3003);


    System.out.print("Tambah 1 simpul di depan: ");

    int data_3003 = 1;

    head_3003 = insertAtFront_3003(head_3003, data_3003);


    printList_3003(head_3003);

```

```

        System.out.print("Tambah 1 simpul di belakang: ");

        int data2_3003 = 7;

        head_3003 = insertAtEnd_3003(head_3003, data2_3003);

        printList_3003(head_3003);

        System.out.print("Tambah 1 simpul ke data 4: ");

        int data3_3003 = 4;

        int pos_3003 = 4;

        head_3003 = insertPos_3003(head_3003, pos_3003, data3_3003);

        printList_3003(head_3003);
    }
}

```

Code diatas merupakan code driver yang menggunakan ADT Node yang sudah dibuat sebelumnya, code ini berfokus pada penambahan node yang akan dihubungkan menggunakan properti next_3003 yang akan menciptakan rantai antar node.

c. **PencarianSLL_2511533003**

```

package pekan5_2511533003;

public class PencarianSLL_2511533003 {

    static boolean searchKey_3003(NodeSLL_2511533003 head_3003, int key_3003) {

```

```

NodeSLL_2511533003 curr_3003 = head_3003;

while (curr_3003 != null) {

    if (curr_3003.data_3003 == key_3003) return true;

    curr_3003 = curr_3003.next_3003;

}

return false;

}

public static void traversal_3003(NodeSLL_2511533003 head_3003) {

    NodeSLL_2511533003 curr_3003 = head_3003;

    while (curr_3003 != null) {

        System.out.print(" " + curr_3003.data_3003);

        curr_3003 = curr_3003.next_3003;

    }

    System.out.println("");

}

public static void main(String[] args) {

    NodeSLL_2511533003 head_3003 = new NodeSLL_2511533003(14);

    head_3003.next_3003 = new NodeSLL_2511533003(21);

    head_3003.next_3003.next_3003 = new NodeSLL_2511533003(13);

    head_3003.next_3003.next_3003.next_3003 = new NodeSLL_2511533003(30);

    head_3003.next_3003.next_3003.next_3003.next_3003 =

        new NodeSLL_2511533003(10);

    System.out.print("Penelusuran SLL : ");

```

```

traversal_3003(head_3003);

int key_3003 = 30;

System.out.print("Cari data " + key_3003 + " = ");

if (searchKey_3003(head_3003, key_3003)) System.out.println("ketemu");

else System.out.println("Tidak Ada");

}

}

```

Code diatas merupakan kode yang berfungsi untuk melakukan pencarian, dikarenakan Node saling terhubung membentuk rantai maka program dapat menelusuri linked-list dari head ke tail.

d. HapusSLL_2511533003

```

package pekan5_2511533003;

public class HapusSLL_2511533003 {

    public static NodeSLL_2511533003 deletedHead_3003(

        NodeSLL_2511533003 head_3003

    ) {

        if (head_3003 == null) return null;

        head_3003 = head_3003.next_3003;

        return head_3003;

    }

    public static NodeSLL_2511533003 removeLastNode_3003(

        NodeSLL_2511533003 head_3003

    ) {

```

```

    if (head_3003 == null) {
        return null;
    }

    if (head_3003.next_3003 == null) {
        return null;
    }

    NodeSLL_2511533003 secondLast_3003 = head_3003;
    while (secondLast_3003.next_3003.next_3003 != null) {
        secondLast_3003 = secondLast_3003.next_3003;
    }

    secondLast_3003.next_3003 = null;

    return head_3003;
}

public static NodeSLL_2511533003 deleteNode_3003(
    NodeSLL_2511533003 head_3003,
    int position_3003
) {
    NodeSLL_2511533003 temp_3003 = head_3003;
    NodeSLL_2511533003 prev_3003 = null;

    if (temp_3003 == null) {
        return head_3003;
    }

```

```
}
```

```
if (position_3003 == 1) {  
    head_3003 = temp_3003.next_3003;  
    return head_3003;  
}
```

```
for (  
    int i_3003 = 1;  
    temp_3003 != null && i_3003 < position_3003;  
    i_3003++  
) {  
    prev_3003 = temp_3003;  
    temp_3003 = temp_3003.next_3003;  
}
```

```
if (temp_3003 != null) {  
    prev_3003.next_3003 = temp_3003.next_3003;  
} else {  
    System.out.println("Data Tidak Ada");  
}
```

```
return head_3003;  
}
```

```
public static void printList_3003(NodeSLL_2511533003 head_3003) {  
    NodeSLL_2511533003 curr_3003 = head_3003;
```

```

while (curr_3003.next_3003 != null) {

    System.out.print(curr_3003.data_3003 + "-->");

    curr_3003 = curr_3003.next_3003;

}

if (curr_3003.next_3003 == null) {

    System.out.println(curr_3003.data_3003);

}

System.out.println();

}

public static void main(String[] args) {

    NodeSLL_2511533003 head_3003 = new NodeSLL_2511533003(1);

    head_3003.next_3003 = new NodeSLL_2511533003(2);

    head_3003.next_3003.next_3003 = new NodeSLL_2511533003(3);

    head_3003.next_3003.next_3003.next_3003 = new NodeSLL_2511533003(4);

    head_3003.next_3003.next_3003.next_3003.next_3003 =

        new NodeSLL_2511533003(5);

    head_3003.next_3003.next_3003.next_3003.next_3003.next_3003 =

        new NodeSLL_2511533003(6);

    System.out.println("list awal: ");

    printList_3003(head_3003);

    head_3003 = deletedHead_3003(head_3003);

```

```

        System.out.println("list setelah head dihapus: ");
        printList_3003(head_3003);

        head_3003 = removeLastNode_3003(head_3003);
        System.out.println("list setelah simpul terakhir dihapus : ");
        printList_3003(head_3003);

        int position_3003 = 2;
        head_3003 = deleteNode_3003(head_3003, position_3003);
        System.out.println("list setelah posisi 2 dihapus : ");
        printList_3003(head_3003);
    }
}

```

Code diatas merupakan code driver yang menggunakan ADT Node dengan tujuan menghapus node pada linked-list, berdasarkan dari konsep linked-list yang memiliki head dan tail serta node diantaranya saling terhubung membentuk rantai, dalam penghapusan Node dapat menggunakan logika yang sama dengan rantai yaitu dapat menghapus bagian head (ujungnya) dengan mudah serta penghapusan di bagian tengah dapat dilakukan dan dihubungkan kembali dengan selayaknya rantai karena Node yang saling bertentangan dapat mengakses satu sama lain.

C. TUGAS

a. Deskripsi Tugas

Pada tugas ini, mahasiswa diminta untuk mengimplementasikan konsep Single Linked List yang bekerja secara dinamis menggunakan pointer (referensi antar node). Tema tugas kali ini adalah Simulasi Antrian Rumah Sakit. Mahasiswa diwajibkan membuat program yang mensimulasikan sistem pendaftaran antrian

pasien, di mana setiap pasien direpresentasikan sebagai sebuah node yang memiliki pointer ke pasien berikutnya. Pasien yang pertama kali mendaftar akan menjadi pasien pertama yang dipanggil (FIFO — First In, First Out).

b. Pengerjaan

```
package pekan5_2511533003;

public class Pasien_2511533003 {

    String name_3003;

    String keluhan_3003;

    int no_antrian_3003;

    Pasien_2511533003 next_3003;

    // Konstruktor

    public Pasien_2511533003(

        String name_3003,

        String keluhan_3003,

        int no_antrian_3003,

        Pasien_2511533003 next_3003

    ) {

        this.name_3003 = name_3003;

        this.keluhan_3003 = keluhan_3003;

        this.no_antrian_3003 = no_antrian_3003;

        this.next_3003 = next_3003;

    }

    // getter

    public String getName_3003() {

        return this.name_3003;

    }

}
```

```

public String getKeluhan_3003() {
    return this.keluhan_3003;
}

public int getNoAntrian_3003() {
    return this.no_antrian_3003;
}

public Pasien_2511533003 getNext_3003() {
    return this.next_3003;
}

// mutator

public void setName_3003(String name_3003) {
    this.name_3003 = name_3003;
}

public void setKeluhan_3003(String keluhan_3003) {
    this.keluhan_3003 = keluhan_3003;
}

public void setNoAntrian_3003(int no_antrian_3003) {
    this.no_antrian_3003 = no_antrian_3003;
}

public void setNext_3003(Pasien_2511533003 next_3003) {
    this.next_3003 = next_3003;
}
}

```

Code diatas merupakan ADT yang digunakan sebagai Node, memiliki data data untuk mendeskripsikan pasien antara lain:

1. name_3003

2. keluhan_3003
3. no_antrian_3003
4. Next_3003

Terkhusus pada next_3003 digunakan untuk point ke node (pasien) lain. Di dalam code juga terdapat konstruktor, getter dan setter yang dapat digunakan.

```
package pekan5_2511533003;

import java.util.Scanner;

public class RumahSakit_2511533003 {

    static int counter_3003 = 1;

    public static Pasien_2511533003 insertAtTail_3003(

        Pasien_2511533003 head_3003

    ) {

        Scanner sc_3003 = new Scanner(System.in);

        System.out.print("Masukkan Nama Pasien: ");

        String name_pasien_3003 = sc_3003.nextLine();

        System.out.print("Masukkan keluhan: ");

        String keluhan_3003 = sc_3003.nextLine();

        int antrian_3003 = counter_3003;

        counter_3003 += 1;

        System.out.println(

            "pasien berhasil di daftarkan! Nomor Antrian " + antrian_3003

        );

        Pasien_2511533003 insert_3003 = new Pasien_2511533003(

            name_pasien_3003,

            keluhan_3003,
```

```

        antrian_3003,

        null

    );

    if (head_3003 == null) {

        return insert_3003;

    }

    Pasien_2511533003 pre_head_3003 = head_3003;

    while (true) {

        if (pre_head_3003.next_3003 == null) {

            pre_head_3003.next_3003 = insert_3003;

            break;

        }

        pre_head_3003 = pre_head_3003.next_3003;

    }

    return head_3003;

}

public static Pasien_2511533003 deleteHead_3003(

    Pasien_2511533003 head_3003

) {

    if (head_3003 == null) {

        System.out.println("pasien kosong");

        return null;

    }

}

```

```

    }

    Pasien_2511533003 pre_head_3003 = head_3003;

    System.out.println(
        "pasien antrian: " + pre_head_3003.getNoAntrian_3003()
    );

    System.out.println("bernama: " + pre_head_3003.getName_3003());
    System.out.println("keluhan: " + pre_head_3003.getKeluhan_3003());
    System.out.println("telah dipanggil");
    return head_3003.next_3003;
}

public static Pasien_2511533003 display_3003(Pasien_2511533003 head_3003) {
    Pasien_2511533003 pre_head_3003 = head_3003;
    if (pre_head_3003 == null) {
        System.out.println("Pasien Kosong");
        return null;
    }
    while (true) {
        if (pre_head_3003 == null) {
            break;
        }
        System.out.println(
            "pasien antrian: " + pre_head_3003.getNoAntrian_3003()
        );
        System.out.println("bernama: " + pre_head_3003.getName_3003());
        System.out.println(
            "keluhan: " + pre_head_3003.getKeluhan_3003() + "\n"
        );
    }
}

```

```

    );

    pre_head_3003 = pre_head_3003.next_3003;
}

return head_3003;
}

public static Pasien_2511533003 search_3003(Pasien_2511533003 head_3003) {
    Pasien_2511533003 pre_head_3003 = head_3003;
    if (pre_head_3003 == null) {
        System.out.println("Pasien Kosong");
        return null;
    }

    Scanner sc_3003 = new Scanner(System.in);

    System.out.print("Masukkan Nama Pasien yang ingin dicari: ");
    String dicari_3003 = sc_3003.nextLine();

    while (true) {
        if (pre_head_3003 == null) {
            System.out.println(
                "pasien bernama " + dicari_3003 + " tidak ditemukan"
            );
            break;
        }
    }
}

```

```

String name_3003 = pre_head_3003.getName_3003();

if (dicari_3003.equalsIgnoreCase(name_3003)) {
    System.out.println(
        "pasien bernama " + dicari_3003 + " ditemukan"
    );
    break;
} else {
    pre_head_3003 = pre_head_3003.next_3003;
}
}

return head_3003;
}

public static Pasien_2511533003 StatusAntrian(Pasien_2511533003 head_3003) {
    Pasien_2511533003 pre_head_3003 = head_3003;
    if (pre_head_3003 == null) {
        System.out.println("Pasien Kosong");
        return null;
    }

    int counting_3003 = 0;
    while (true) {
        if (pre_head_3003 == null) {
            break;

```

```

    }

    counting_3003 += 1;

    pre_head_3003 = pre_head_3003.next_3003;
}

System.out.println("Panjang Antrean => " + counting_3003);
System.out.println("pasien antrian: " + head_3003.getNoAntrian_3003());
System.out.println("Informasi Pasien Terdepan");
System.out.println("bernama: " + head_3003.getName_3003());
System.out.println("keluhan: " + head_3003.getKeluhan_3003() + "\n");
return head_3003;
}

public static void main(String[] args) {
    Pasien_2511533003 head_3003 = null;
    Scanner sc_3003 = new Scanner(System.in);
    while (true) {
        System.out.println(
            "\n=== Antrian Rumah Sakit NIM: 2511533003 ==="
        );
        System.out.println("1. Daftarkan Pasien    (Insert)");
        System.out.println("2. Panggil Pasien    (Delete Head)");
        System.out.println("3. Tampilkan Antrian    (Display)");
        System.out.println("4. Cari Pasien        (Search)");
        System.out.println("5. Cek Status Antrian ");
        System.out.println("6. Keluar ");
    }
}

```



```

        System.out.print("pilihan: ");

        int opsi_3003 = sc_3003.nextInt();

        System.out.println();

        if (opsi_3003 == 1) {
            head_3003 = insertAtTail_3003(head_3003);
        } else if (opsi_3003 == 2) {
            head_3003 = deleteHead_3003(head_3003);
        } else if (opsi_3003 == 3) {
            head_3003 = display_3003(head_3003);
        } else if (opsi_3003 == 4) {
            head_3003 = search_3003(head_3003);
        } else if (opsi_3003 == 5) {
            head_3003 = StatusAntrian(head_3003);
        } else if (opsi_3003 == 6) {
            break;
        } else {
            System.out.println("opsi tidak tersedia di pilihan");
        }
    }
    sc_3003.close();
}
}

```

Code diatas merupakan code driver yang menggunakan ADT pasien yang telah dibuat sebelumnya, saat dijalankan pertama kali akan menampilkan opsi yang dapat dipilih oleh user.

```

=== Antrian Rumah Sakit NIM: 2511533003 ===
1. Daftarkan Pasien      (Insert)
2. Panggil Pasien        (Delete Head)
3. Tampilkan Antrian     (Display)
4. Cari Pasien           (Search)
5. Cek Status Antrian
6. Keluar
pilihan: █

```

Untuk memasukkan data pasien yang akan mengantri, dapat menggunakan pilihan 1, User juga diminta untuk mengisi nama penyakit serta keluhan yang diderita.

```

=== Antrian Rumah Sakit NIM: 2511533003 ===
1. Daftarkan Pasien      (Insert)
2. Panggil Pasien        (Delete Head)
3. Tampilkan Antrian     (Display)
4. Cari Pasien           (Search)
5. Cek Status Antrian
6. Keluar
pilihan: 1

Masukkan Nama Pasien: yazem
Masukkan keluhan: pusing
pasien berhasil di daftarkan! Nomor Antrian 1

=== Antrian Rumah Sakit NIM: 2511533003 ===
1. Daftarkan Pasien      (Insert)
2. Panggil Pasien        (Delete Head)
3. Tampilkan Antrian     (Display)
4. Cari Pasien           (Search)
5. Cek Status Antrian
6. Keluar
pilihan: 1

Masukkan Nama Pasien: faiz anri
Masukkan keluhan: sakit kepala
pasien berhasil di daftarkan! Nomor Antrian 2

=== Antrian Rumah Sakit NIM: 2511533003 ===
1. Daftarkan Pasien      (Insert)
2. Panggil Pasien        (Delete Head)
3. Tampilkan Antrian     (Display)
4. Cari Pasien           (Search)
5. Cek Status Antrian
6. Keluar
pilihan: 1

Masukkan Nama Pasien: marcello bayu
Masukkan keluhan: migrain
pasien berhasil di daftarkan! Nomor Antrian 3

=== Antrian Rumah Sakit NIM: 2511533003 ===
1. Daftarkan Pasien      (Insert)
2. Panggil Pasien        (Delete Head)
3. Tampilkan Antrian     (Display)
4. Cari Pasien           (Search)
5. Cek Status Antrian
6. Keluar
pilihan: █

```

Setelah data pasien dimasukkan, user dapat melihat data pasien melalui opsi 3, yang mana linked-list yang tercipta akan dijelajahi dari awal hingga akhir untuk mencetak pasien yang terdaftar.

```
=== Antrian Rumah Sakit NIM: 2511533003 ===
1. Daftarkan Pasien      (Insert)
2. Panggil Pasien        (Delete Head)
3. Tampilkan Antrian     (Display)
4. Cari Pasien           (Search)
5. Cek Status Antrian
6. Keluar
pilihan: 3

pasien antrian: 1
bernama: yazem
keluhan: pusing

pasien antrian: 2
bernama: faiz anri
keluhan: sakit kepala

pasien antrian: 3
bernama: marcello bayu
keluhan: migrain
```

Jika pasien dipanggil, maka akan mengikuti FIFO yang mana pasien yang masuk mengantre duluan akan keluar (dipanggil) duluan, bisa menggunakan opsi 2.

```

=== Antrian Rumah Sakit NIM: 2511533003 ===
1. Daftarkan Pasien      (Insert)
2. Panggil Pasien        (Delete Head)
3. Tampilkan Antrian     (Display)
4. Cari Pasien           (Search)
5. Cek Status Antrian
6. Keluar
pilihan: 2

pasien antrian: 1
bernama: yazem
keluhan: pusing
telah dipanggil

=== Antrian Rumah Sakit NIM: 2511533003 ===
1. Daftarkan Pasien      (Insert)
2. Panggil Pasien        (Delete Head)
3. Tampilkan Antrian     (Display)
4. Cari Pasien           (Search)
5. Cek Status Antrian
6. Keluar
pilihan: 3

pasien antrian: 2
bernama: faiz anri
keluhan: sakit kepala

pasien antrian: 3
bernama: marcello bayu
keluhan: migrain

```

Bisa dilihat bahwa pasien antrian 1 telah dipanggil menyisakan 2 pasien lagi, untuk memperbanyak contoh akan ditambahkan 1 pasien lagi setelah itu menggunakan opsi 4 untuk mencari salah satu pasien dalam antrian.

```

=== Antrian Rumah Sakit NIM: 2511533003 ===
1. Daftarkan Pasien      (Insert)
2. Panggil Pasien        (Delete Head)
3. Tampilkan Antrian      (Display)
4. Cari Pasien            (Search)
5. Cek Status Antrian
6. Keluar
pilihan: 1

Masukkan Nama Pasien: hasan faqot
Masukkan keluhan: mual
pasien berhasil di daftarkan! Nomor Antrian 4

=== Antrian Rumah Sakit NIM: 2511533003 ===
1. Daftarkan Pasien      (Insert)
2. Panggil Pasien        (Delete Head)
3. Tampilkan Antrian      (Display)
4. Cari Pasien            (Search)
5. Cek Status Antrian
6. Keluar
pilihan: 3

pasien antrian: 2
bernama: faiz anri
keluhan: sakit kepala

pasien antrian: 3
bernama: marcello bayu
keluhan: migrain

pasien antrian: 4
bernama: hasan faqot
keluhan: mual

=== Antrian Rumah Sakit NIM: 2511533003 ===
1. Daftarkan Pasien      (Insert)
2. Panggil Pasien        (Delete Head)
3. Tampilkan Antrian      (Display)
4. Cari Pasien            (Search)
5. Cek Status Antrian
6. Keluar
pilihan: 4

Masukkan Nama Pasien yang ingin dicari: marcello bayu
pasien bernama marcello bayu ditemukan

=== Antrian Rumah Sakit NIM: 2511533003 ===
1. Daftarkan Pasien      (Insert)
2. Panggil Pasien        (Delete Head)
3. Tampilkan Antrian      (Display)
4. Cari Pasien            (Search)
5. Cek Status Antrian
6. Keluar
pilihan: 

```

Pasien ditambahkan lalu melakukan pencarian pasien yang bernama marcello bayu, bisa dilihat pada output diatas menampilkan keluaran bahwa marcello bayu ditemukan di dalam antrean, jika mencari nama yang tidak ada dalam antrean akan menghasilkan output seperti yang ada dibawah ini.

```
=== Antrian Rumah Sakit NIM: 2511533003 ===
1. Daftarkan Pasien      (Insert)
2. Panggil Pasien        (Delete Head)
3. Tampilkan Antrian     (Display)
4. Cari Pasien           (Search)
5. Cek Status Antrian
6. Keluar
pilihan: 4

Masukkan Nama Pasien yang ingin dicari: yazem
pasien bernama yazem tidak ditemukan
```

Akan dihasilkan output yazem tidak ada dalam antrean. Terdapat satu opsi lagi yaitu opsi ke-5 yang akan mencetak panjang dari antrean dan memberikan informasi pasien paling atas atau yang menjadi head.

```
=== Antrian Rumah Sakit NIM: 2511533003 ===
1. Daftarkan Pasien      (Insert)
2. Panggil Pasien        (Delete Head)
3. Tampilkan Antrian     (Display)
4. Cari Pasien           (Search)
5. Cek Status Antrian
6. Keluar
pilihan: 5

Panjang Antrean => 3
pasien antrian: 2
Informasi Pasien Terdepan
bernama: faiz anri
keluhan: sakit kepala
```

Untuk kasus antrian kosong, sebagai berikut

```
=== Antrian Rumah Sakit NIM: 2511533003 ===
1. Daftarkan Pasien      (Insert)
2. Panggil Pasien        (Delete Head)
3. Tampilkan Antrian     (Display)
4. Cari Pasien           (Search)
5. Cek Status Antrian
6. Keluar
pilihan: 2

pasien kosong

=== Antrian Rumah Sakit NIM: 2511533003 ===
1. Daftarkan Pasien      (Insert)
2. Panggil Pasien        (Delete Head)
3. Tampilkan Antrian     (Display)
4. Cari Pasien           (Search)
5. Cek Status Antrian
6. Keluar
pilihan: 3

Pasien Kosong

=== Antrian Rumah Sakit NIM: 2511533003 ===
1. Daftarkan Pasien      (Insert)
2. Panggil Pasien        (Delete Head)
3. Tampilkan Antrian     (Display)
4. Cari Pasien           (Search)
5. Cek Status Antrian
6. Keluar
pilihan: 4

Pasien Kosong

=== Antrian Rumah Sakit NIM: 2511533003 ===
1. Daftarkan Pasien      (Insert)
2. Panggil Pasien        (Delete Head)
3. Tampilkan Antrian     (Display)
4. Cari Pasien           (Search)
5. Cek Status Antrian
6. Keluar
pilihan: 5

Pasien Kosong

=== Antrian Rumah Sakit NIM: 2511533003 ===
1. Daftarkan Pasien      (Insert)
2. Panggil Pasien        (Delete Head)
3. Tampilkan Antrian     (Display)
4. Cari Pasien           (Search)
5. Cek Status Antrian
6. Keluar
pilihan: █
```

BAB 3

PENUTUPAN

a. Kesimpulan

Dengan penulisan code linked list ini dapat dipahami adalah linked list merupakan struktur data yang sangatlah berguna untuk kasus yang memerlukan relasi antar data dengan mempertahankan urutan, tanpa adanya size tetap membuat struktur ini sangatlah fleksibel dan berguna dalam kasus yang memiliki size yang tidak pasti.

b. Saran

Laporan praktikum yang telah penulis tuliskan, masih memiliki kekurangan. Penulis sendiri pun juga membuka saran dan kritikan untuk meningkatkan kualitas laporan pratikum ini.

DAFTAR PUSTAKA

Oracle. *Class LinkedList*. Java Platform SE 8 Documentation. Diakses dari [Oracle Java Documentation](#) pada 6 Mei 2026.